

ROS (Robot Operating System)



Que es ROS?

ROS (Robot Operating System) es un conjunto de herramientas software y librerías que agilizan la construcción de sistemas robóticos. ROS provee los servicios estándar de un sistema operativo tales como abstracción del hardware, control de dispositivos de bajo nivel, paso de mensajes entre procesos, entre otros. Está basado en una arquitectura de grafos donde el procesamiento toma lugar en los nodos que pueden recibir, mandar y multiplexar mensajes de sensores, control, estados, entre otros. Se encuentra orientado para un sistema UNIX (Ubuntu (Linux)) aunque también se está adaptando a otros sistemas operativos. ROS es software libre bajo términos de licencia BSD. Esta licencia permite libertad para uso comercial e investigación.

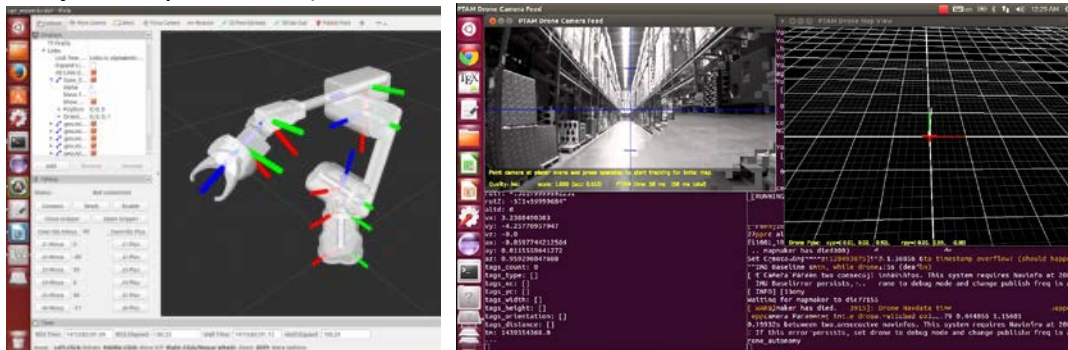


Por qué usar ROS?

Aprender a utilizar un entorno de trabajo demanda tiempo y esfuerzo, por esto resulta necesario remarcar que ventajas nos trae el uso de ROS, y en qué casos conviene su utilización. ROS fue y es pensado para ser un conjunto de herramientas aplicables en entornos robóticos (no restringido a un tipo de robot en particular) con el objetivo de no reinventar la rueda. Conociendo esto existen algunos puntos de la implementación de ROS que lo hacen atractivo:



- **Sistema distribuido:** la estructura de ROS permite la comunicación entre múltiples procesos que pueden o no trabajar en la misma computadora. Por ejemplo, en robots complejos puede existir un proceso que maneje la información de un sensor en particular, otro que se ocupe de la odometría, otro del control de velocidad, etc. Esta segmentación del trabajo por un lado facilitará la depuración de la programación, y a su vez como todos estos procesos no necesariamente se encontrarán en la misma PC, nos habilitan a realizar tareas de colaboración o servicios a distancia.
- **Reutilización de código:** el crecimiento de aplicaciones robóticas conlleva a un crecimiento aun mayor de algoritmos aplicados en ellas. Por ejemplo, navegación, evitación de obstáculos, mapeo, etc. ROS define un estándar para la estructura de estos procesos y la comunicación entre ellos. Al realizar esto permite la exportación de un código de un sistema a otro idealmente sin modificaciones.
- **Rápida prueba de conceptos:** uno de los objetivos del sistema y su estandarización de código es el de poder desarrollar por un lado un software de alto nivel (del que se preocupa más ROS) y por otro el código de bajo nivel y hardware. Esta separación nos permitirá mediante herramientas de simulación probar nuestros códigos (reemplazando el bajo nivel y hardware) sin necesidad de utilizar con un robot real.



- **Almacenamiento de datos:** ROS provee servicios de captura de datos y visualización, los cuales nos permitirán saber que sucedió (bien o mal) en un ensayo experimental llevando estos datos a un entorno simulado, pudiendo así observar la evolución de cada variable de nuestro sistema.

Como comenzar?

Para comenzar debemos tener instalado nuestro sistema ROS, si vamos a la página oficial <http://www.ros.org/> encontraremos distintas versiones del sistema, ROS1 y ROS2. Fundamentalmente difieren en la arquitectura de base utilizada (ROS 1 utiliza una estructura Maestro-esclavo y el middleware XML-RPC, mientras que ROS 2 utiliza Data Distribution Service (DDS)), características y herramientas varias del ecosistema ROS.

A su vez cada una de estas versiones cuenta con diferentes distribuciones, una distribución ROS es un conjunto versionado de paquetes ROS. Son similares a las distribuciones de Linux (por ejemplo, Ubuntu). El propósito de las distribuciones ROS es permitir a los desarrolladores



trabajar con una base de código relativamente estable. Cada distribución implementa mejoras respecto de las anteriores y asegura una cierta actualización de los paquetes incluidos en la versión “básica” de ROS, dejando fuera códigos de usuarios, por tanto puede que encontremos códigos aun de mismas versiones de ROS pero de diferentes distribuciones que no sean compatibles con la distribución que deseemos, pero que con algo de trabajo los podremos migrar.

Con que versión y distribución trabajar?

Para responder lo anterior, debemos analizar sobre que trabajaremos, en el caso de un proyecto nuevo lo recomendado es trabajar sobre la versión de ROS2, luego deberemos analizar un poco la aplicación particular y paquetes que nos interesen para elegir que distribución será la mas adecuada (o la que nos simplifique nuestro trabajo). En este curso abordaremos la versión de ROS2 distribución Jazzy Jalisco, ya que es la mas actual de esta versión.

La instalación de ROS2, podemos realizar siguiendo los pasos en su documentación:

<https://docs.ros.org/en/jazzy/index.html>

De allí podremos observar que podemos instalarlo de distintas formas y en distintos sistemas operativos, para este curso utilizaremos el sistema Ubuntu 24.04 y utilizaremos la versión desktop (existen siempre dos versiones de instalación de las distribuciones ROS la base y la desktop, esta última agrega sobre la primera las aplicaciones que utilizan interfaz de usuario).

Nota: Anexo a este documento se presentara una guía para utilizar ROS2 en un contenedor, si bien esta opción presenta limitaciones en cuanto performance de ejecución, para nuestros primeros intentos en este sistema nos ofrece un espacio de trabajo seguro y fácilmente restaurable.

Nota: Los comandos que presentaremos en este documento solo funcionan para ROS2, aquellos interesados en saber cómo funciona ROS1 pueden consultar su documentación en: <https://wiki.ros.org/Documentation>

Nota: ROS1 vs. ROS2, si recién nos iniciamos en el mundo de ROS encontraremos dos versiones. ROS 1, desde la original creada por Willow Garage en 2007 hasta Noetic con soporte (en lo planeado al menos) hasta 2025, y una nueva versión ROS2.

La justificación de estas dos versiones viene de la mano de la extensión de ROS al campo industrial donde se necesita aplicaciones de tiempo real, certificaciones, seguridad, etc. que



implicaban una modificación muy importante de la estructura de ROS1, por lo que directamente se comenzó desde cero con una nueva versión.

- **Primer ejemplo:**

Antes de comenzar con la estructura de ROS, realizaremos un mínimo ejemplo usando el simulador turtlesim, el objetivo aquí es comprobar la buena instalación de ROS, conocer el simulador y obtener una idea inicial de la forma de trabajo de ROS.

Abriremos dos consolas de trabajo, en cada una ejecutaremos:

```
source /opt/ros/jazzy/setup.bash
```



Nota: Este comando indicara al sistema donde buscar el path a la instalación de ROS. Esto lo deberemos hacer con cada nueva consola que utilizamos, si deseamos evitarlo y que se agregue de forma automática podemos utilizar el comando:

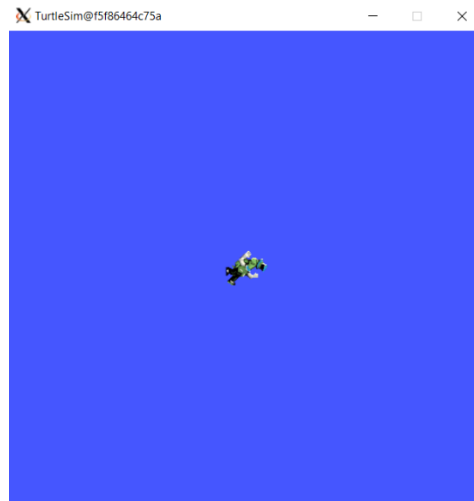
```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
```

Luego ejecutamos una de las siguientes líneas de comando en cada una:

```
ros2 run turtlesim turtlesim_node
```

```
ros2 run turtlesim turtle_teleop_key
```

La primera línea iniciará el simulador y la segunda iniciará un programa (nodo) que se comunicará con el simulador a través del cual usando las flechas de teclado podremos mover la Tortuga del simulador (debemos tener activa la terminal donde está corriendo este nodo para que detecte las teclas):



```

root@f5f86464c75a: ~/ros2_ws 95x8
root@f5f86464c75a:~/ros2_ws# ros2 run turtlesim turtle_teleop_key
Reading from keyboard
-----
Use arrow keys to move the turtle.
Use g|b|v|c|d|e|r|t keys to rotate to absolute orientations. 'f' to cancel a rotation.
'q' to quit.

```

Con este ejemplo podemos abordar los siguientes temas:

Paquetes (packages):

Todo el software de ROS está organizado en paquetes. Un paquete de ROS es una colección coherente de archivos, que en general incluyen ejecutables y archivos de soporte que sirven a un propósito específico. En el ejemplo anterior utilizamos dos ejecutables `turtlesim_node` y `turtle_teleop_key`, ambos pertenecientes al paquete `turtlesim`.

ROS incluye una serie de herramientas para trabajar con los paquetes:

ros2 pkg list	Nos da la lista de todos los paquetes ROS instalados
ros2 pkg prefix package-name	Nos servirá para encontrar el directorio de un paquete en particular



- A lo largo del apunte iremos nombrando herramientas de ROS en forma genérica, de forma que si se generan dudas sea más fácil de ser googleadas para aclarar su funcionamiento. En la tabla anterior por ejemplo cuando nos



referimos a **package-name**, al ejecutar esta línea deberemos cambiar esto por el nombre del paquete de interés.

• También podremos consultar la documentación de ROS2 Jazzy en: <https://docs.ros.org/en/jazzy/index.html>

Nodos:

Un nodo en ROS es una instancia de un programa. Es importante destacar el concepto de **instancia**, por ejemplo, podremos ejecutar múltiples copias del mismo programa a la vez, pero si deseamos comportamientos distintos de los mismos deberemos nombrar cada una con un nombre de nodo diferente (esto se recomienda para evitar conflictos entre nodos corriendo en un mismo **namespace** o **domain ID** – formas de agrupar nodos de ROS2). Cada una de estas copias es tratada como un nodo independiente, es decir una ejecución independiente.

En el ejemplo de turtlesim, creamos dos nodos:

- Un nodo es una instancia del ejecutable llamado **turtlesim_node**. Este nodo es el responsable de crear la ventana del turtlesim y la simulación de los movimientos de la tortuga.
- El otro nodo es una instancia de un ejecutable llamado **turtle_teleop_key**. Este nodo espera por la acción de las flechas de teclado, convirtiendo estas en comandos de movimientos a ser enviados al nodo turtlesim_node.

ROS incluye una serie de herramientas para trabajar con nodos:

ros2 run package-name executable-name	Crea un nodo (o dicho de otra forma ejecuta un programa de ROS). Requiere de dos parámetros el nombre del paquete y un ejecutable que pertenezca al mismo.
ros2 node list	Crea una lista de los nodos activos
ros2 node info node-name	Sirve para obtener información de un nodo. Este comando nos devolverá la lista de tópicos en los cuales el nodo esta suscrito y en cual actúa como publicador, los servicios ofrecidos por el nodo y un resumen de las conexiones que ha hecho con otros nodos.

Por ejemplo, si ejecutamos este `ros2 node list` con el ejemplo de turtlesim devolverá:



```
root@f5f86464c75a: ~/ros2_ws 95x8
root@f5f86464c75a:~/ros2_ws# ros2 node list
/teleop_turtle
/turtlesim
root@f5f86464c75a:~/ros2_ws#
```

/turtlesim será el nodo del simulador y /teleop_turtle el nodo de comando. Resulta interesante ver que los nombres de los nodos no necesariamente coinciden con el nombre de su correspondiente ejecutable.

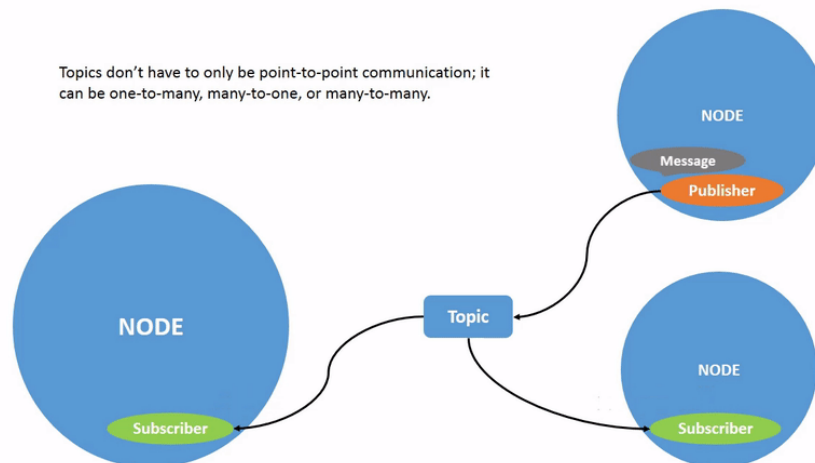


Si quisiéramos darle un nombre en particular a un nodo podríamos utilizar la opción **remap**, en donde **node-name** será el nombre que le quisiéramos dar:

```
ros2 run package-name executable-name --ros-args --remap __node:=node-name
```

Tópicos y mensajes:

El principal mecanismo que posee ROS para comunicar nodos es el uso de mensajes (estructuras de datos estandarizadas para el intercambio de información). Los mensajes en ROS están organizados en tópicos (**topics**). La idea es que un nodo que desee compartir información publicará mensajes en un apropiado tópico o tópicos; un nodo que desee recibir información se suscribirá al tópico o tópicos en los que se encuentre la información buscada. Una vez realizada la suscripción de ambos, el mensaje al ser publicado es enviado directamente al suscriptor/es.

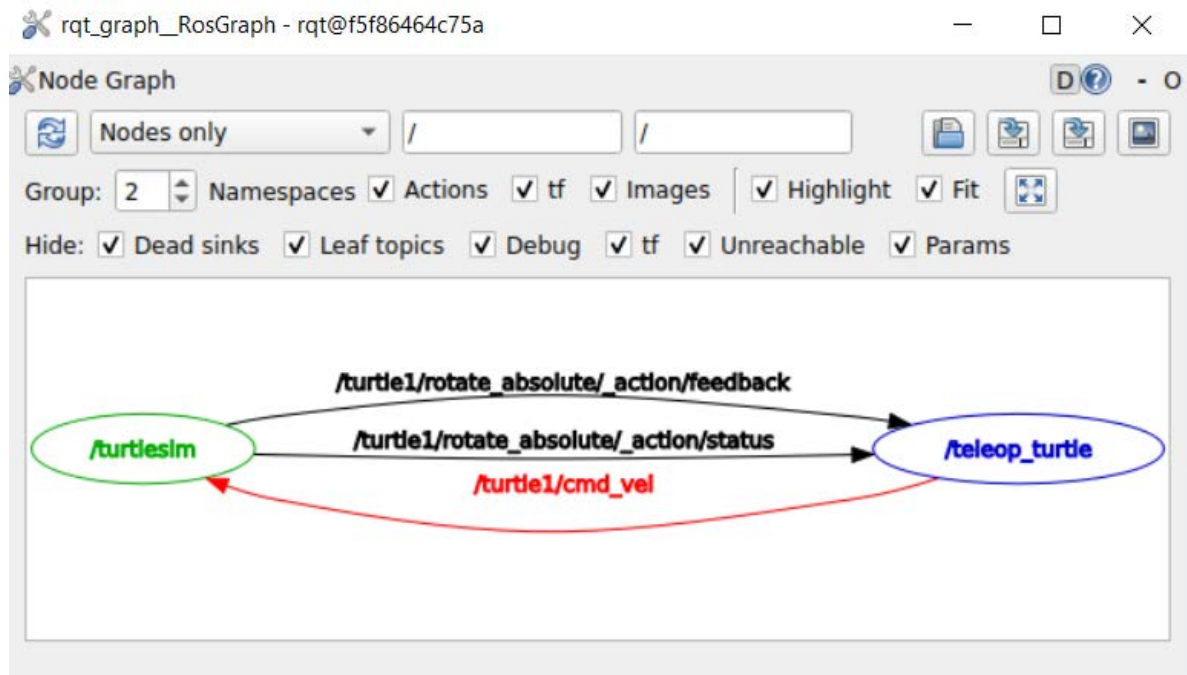




rqt_graph es una herramienta de ROS que permite ver fácil y rápidamente la relación entre nodos y tópicos, si ejecutamos este comando:

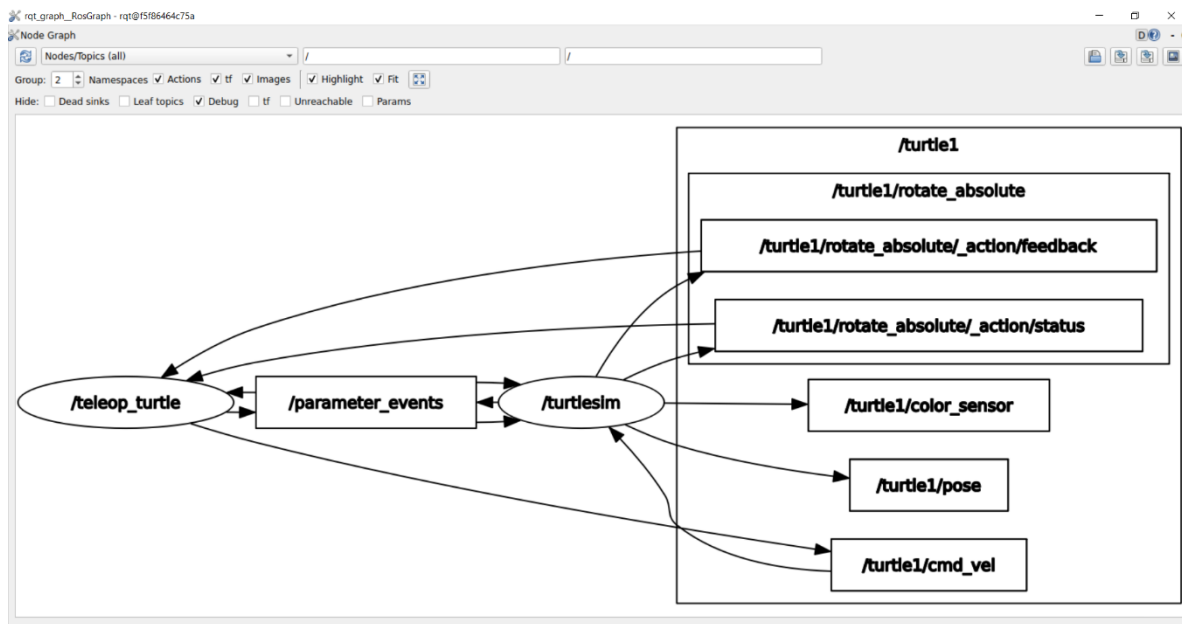
```
rqt_graph
```

con el ejemplo del turtlesim nos devolverá:



En este gráfico, los óvalos representan nodos, y las flechas representan tópicos o acciones (otra forma de comunicación en ROS). En el caso de los tópicos las flechas indican la relación publicador-suscriptor. En el gráfico vemos (ver tópico en rojo) que un nodo llamado `/teleop_turtle` publica mensajes en un tópico llamado `/turtle1/cmd_vel`, y que un nodo llamado `/turtlesim` está suscrito a estos mensajes.

Además, `rqt_graph` permite visualizar este grafico de distintas maneras las cuales podemos probar cambiando las casillas marcada o los menús que aparecen en su ventana. Una opción conveniente es: **Nodes/Topics(all)** y desmarcar todas las casillas excepto **Debug**:



En esta configuración podemos ver que el nodo turtlesim además de estar suscrito a comandos de velocidad también publica su posición y datos sobre su sensor simulado de detección de color.

Notar que:

- Al simulador no le interesa quien publique los comandos de velocidad, por lo que cualquier nodo publicador que publique en el tópico puede controlar la tortuga.
- Por otro lado al nodo de tele operación no le interesa quien está suscrito a sus mensajes, por lo que cualquier nodo suscrito a ese tópico podrá recibir sus mensajes.

ROS incluye una serie de herramientas para trabajar con tópicos y mensajes:

ros2 topic list	Nos devuelve una lista de los tópicos activos.
ros2 topic list -t	Nos devuelve una lista de los tópicos activos, y encerrado entre corchetes su tipo
ros2 topic echo topic-name	En la terminal en que se ejecute nos mostrara los mensajes que se publiquen a partir de su ejecución.
ros2 topic hz topic-name	Nos dice la velocidad con que los mensajes son publicados en el tópico.
ros2 topic bw topic-name	Nos dice el ancho de banda consumido por los mensajes publicados en el tópico.
ros2 topic info topic-name	Sirve para obtener información del tópico.

Siguiendo con el ejemplo de turtlesim, si obtenemos información del tópico de color usando:



```
ros2 topic info /turtle1/color_sensor
```

Obtenemos:

```
root@f5f86464c75a: ~ 83x10
root@f5f86464c75a:~# ros2 topic info /turtle1/color_sensor
Type: turtlesim/msg/Color
Publisher count: 1
Subscription count: 0
root@f5f86464c75a:~#
```

De lo anterior centraremos nuestra atención en la primera línea, la cual nos muestra el tipo de mensaje en ese tópic. En este caso para el topico **/turtle1/color_sensor**, el tipo de mensaje es **turtlesim/msg/Color**. Aquí por tipo entendemos tipo de dato que compone el mensaje. El tipo de mensaje define como se organiza cada mensaje, y que tipo de variables o datos encontramos en él.



Al igual que con el nombre de los nodos, si quisiéramos cambiar el nombre de un topico, y darle un nombre en particular podríamos utilizar la opcion **remap**, por ejemplo para cambiar el nombre del topico del sensor del ejemplo anterior al lanzar el nodo podríamos usar:

```
ros2 run turtlesim turtlesim_node --ros-args --remap /turtle1/color_sensor:=/nuevo_nombre_topico
```

Como todo en ROS, cada mensaje pertenece a un paquete específico, el nombre de los mensajes siempre contiene una barra, la parte previa a esta indica el paquete del cual es parte. Esta configuración es útil para entender el contenido del mensaje y evitar colisiones con mensajes del mismo nombre, pero de paquetes distintos.

ros2 interface show message-type	Nos dice las características de un mensaje en particular
---	--

Por ejemplo, si deseamos ver el detalle del tipo **turtlesim/msg/Color**

```
ros2 interface show turtlesim/msg/Color
```

Obtenemos:



```
root@f5f86464c75a: ~ 83x10
root@f5f86464c75a:~# ros2 interface show turtlesim/msg/Color
uint8 r
uint8 g
uint8 b
root@f5f86464c75a:~#
```

Este resultado nos muestra la estructura del tipo de mensaje en particular. Aquí una serie de unsigned int con los nombres r g b. Estos representan la intensidad de los colores rojo verde y azul del pixel debajo de la tortuga del simulador.

Otro ejemplo de tipo de mensaje es **geometry_msgs/Twist**. Este es el tipo de mensaje del tópico /turtle1/cmd_vel:

```
root@f5f86464c75a: ~ 83x23
root@f5f86464c75a:~# ros2 topic info /turtle1/cmd_vel
Type: geometry_msgs/msg/Twist
Publisher count: 1
Subscription count: 1
root@f5f86464c75a:~# ros2 interface show geometry_msgs/msg/Twist
# This expresses velocity in free space broken into its linear and angular parts.

Vector3  linear
  float64 x
  float64 y
  float64 z
Vector3  angular
  float64 x
  float64 y
  float64 z
root@f5f86464c75a:~#
```

En este caso el mensaje está compuesto por los campos linear (velocidad lineal) y angular (velocidad angular). Vemos que ambos son datos de tipo **Vector3**. La indentación muestra que los campos llamados x,y,z son miembros de los dos campos de mayor nivel. Cada uno de estos campos por otra parte son floats de 64 bits.

Para publicar mensajes por la línea de comando podemos usar el siguiente comando, donde el argumento '**<args>**' son los datos que debemos pasarle al tópico, en la estructura del tipo de mensaje particular. Es importante notar que esta estructura en sintaxis YAML:

```
ros2 topic pub topic_name msg_type '<args>'
```



Por ejemplo, si deseamos publicar una velocidad para que la Tortuga del simulador se mueva podremos usar:

```
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

Esto publica una velocidad lineal de 2m/s en la dirección x por una única vez (esto último lo indicamos con la opción **–once**).

Suele ser necesario tener una publicación de comandos cada cierta frecuencia, por ejemplo, en nuestro caso para mantener la tortuga en movimiento esto lo podemos realizar mediante la opción **–rate** la cual se acompaña con la frecuencia a la que se realizaran las publicaciones. Por ejemplo, el siguiente comando realiza la misma operación que antes, pero la publicación se realiza de forma continuada a una frecuencia de 1 hz:

```
ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

Actividad



- 1) Mate los nodos que pudieran estar corriendo y corra dos nodos `turtlesim_node` y dos nodos `turtle_teleop_key`. Para lo anterior primero piense las siguientes cuestiones:
 - ¿ROS acepta varios nodos con el mismo nombre?
 - ¿Qué pasa si ejecuta `rqt_graph`?
 - ¿Y si ejecuta `ros2 node list`?
- 2) Repita la operación de lanzar los nodos, pero todos con nombres diferentes.
 - ¿Qué tortuga esperamos se mueva con cada teleop?
 - ¿Cómo espera sea el grafico de nodos y tópicos?
- 3) Si se desea que cada teleop controle en forma independiente cada tortuga, ¿Qué modificación podría realizar?
 - ¿Cómo espera sea el grafico de nodos y tópicos, en este caso?
- 4) De los ensayos anteriores que conclusiones puede alcanzar respecto a:
 - ¿Cómo definiría la comunicación por tópicos?
 - ¿Cómo diría que es el acoplamiento entre nodos?



Hasta ahora vimos que al ejecutar un nodo necesariamente ocupamos un terminal, mas adelante veremos como ejecutar varios nodos desde un único terminal usando los archivos `launchfile`. Como atajo rápido ahora podemos

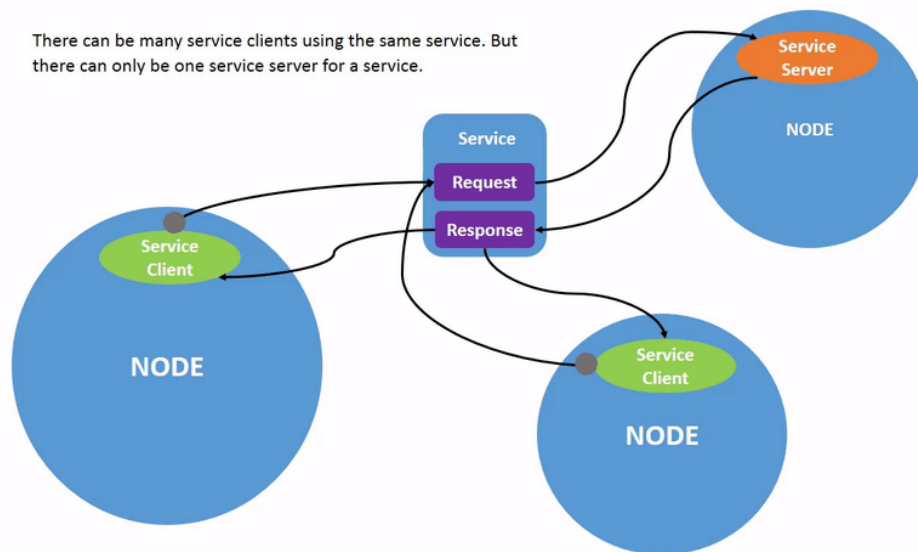


usar cualquier emulador de terminales que nos permita un orden adecuado de las terminales abiertas, por ejemplo terminator. Para instalarlo use: `sudo apt-get install terminator`.

No toda la comunicación en ROS se realiza mediante tópicos, también existen:

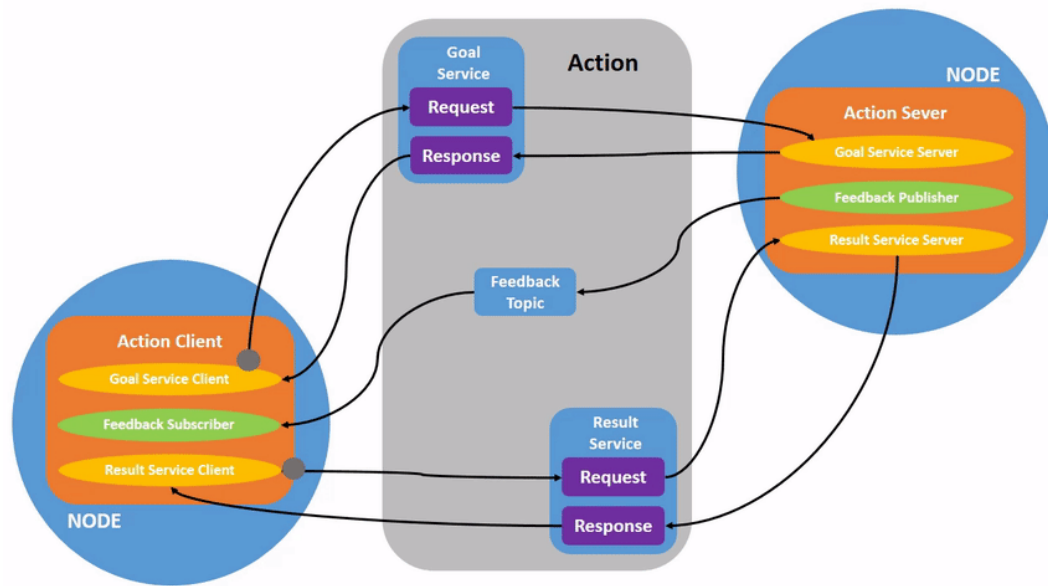
- **Servicios**, que se trata de una comunicación basada en el formato cliente-servidor, un nodo presenta un servicio al cual se puede acceder o demandar desde otro nodo, produciendo algún tipo de respuesta. A diferencia de la publicación en tópicos, aquí hay una interacción entre los nodos participantes, este tipo de comunicación suele reservarse para actividades de cambio de información o acciones asincrónicas. El siguiente tutorial es un buen punto para ver el funcionamiento de este tipo de comunicación:

<https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html>



- **Acciones**, en principio son similares a la comunicación por tópicos, pero están pensadas para acciones de mayor duración. Esta forma de comunicación se basa en el pedido de una acción (de un nodo a otro), durante el transcurso de la misma el nodo que “acciona” puede devolver información del estado de la acción y al terminar un mensaje de finalización de la misma. El siguiente tutorial es un buen punto para ver el funcionamiento de este tipo de comunicación:

<https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html>



- **Primer programa: hola mundo!**

Hasta el momento hemos utilizado código pre-existente en ROS. A continuación, analizaremos un pequeño código que realizara el conocido *hello world!*, con el que todos hemos comenzado a programar.

Nota: El código presentado aquí es en lenguaje Python, ROS nos permite trabajar tanto con Python como con C++ en forma indistinta. Es decir, podremos tener algunos nodos escritos en Python y otros en C++ y el sistema funcionará igual. La estructura de trabajo será la misma en ambos casos, lo que difiere un poco es la forma de organizar los paquetes entre cada lenguaje.

Crear un workspace (espacio de trabajo) y un paquete

Como hemos visto toda la programación de ROS se basa en paquetes, los paquetes que creemos se encontraran dentro de un espacio de trabajo. Por lo tanto, para comenzar a trabajar en ROS será necesario primero crear nuestro espacio de trabajo, para luego crear nuestros paquetes dentro del mismo.

A la hora de crear paquetes ROS nos ofrece una herramienta llamada **colcon**, esta nos permitirá la rápida síntesis de paquetes y en una forma ordenada. Un espacio de trabajo en general tendrá la forma:



```
workspace_folder/      -- directorio espacio de trabajo
  src/                  -- carpeta source donde se encontraran el codigo de nuestro paquetes
  cpp_package_1/        -- Carpeta de paquete lenguaje C++
    CMakeLists.txt
    include/cpp_package_1/
    package.xml
    src/                -- carpeta source de codigo del paquete

  py_package_1/         --Carpeta de paquete lenguaje Python
    package.xml
    resource/py_package_1
    setup.cfg
    setup.py
    py_package_1/
  ...
  cpp_package_n/
    CMakeLists.txt
    include/cpp_package_n/
    package.xml
    src/
```

Analizando particularmente la forma de un paquete Python encontramos:

- **package.xml** archivo que contendrá metadatos sobre el paquete
- **resource/<package_name>** marker file para el paquete (este archivo controla la generación de ejecutables y otros archivos – que no son códigos source)
- **setup.cfg** es requerido cuando el paquete tiene ejecutables, sirve para que ros2 run pueda encontrarlos
- **setup.py** contiene instrucciones de como instalar el paquete (puntos de acceso)
- **<package_name>** - **Directorio con el mismo nombre que el paquete**, utilizado por las herramientas de ROS 2 para encontrar el paquete, contiene el archivo `__init__.py` , y los archivos `.py` que generemos para nuestros nodos



Nuestro espacio de trabajo puede estar localizado en cualquier parte del disco, aquí por ejemplo llamaremos a la carpeta de trabajo **ros2_ws** y se encontrara en el root. Para crear nuestro espacio de trabajo realizamos:

```
mkdir -p ~/ros2_ws/src  
cd ~/ros2_ws
```

Nota – algunos comandos útiles de Linux -:

Para movernos entre carpetas desde terminal podemos usar el comando **cd** de Linux.

```
cd nombre_carpeta
```

Si queremos volver a un nivel anterior de la raíz de directorios podemos usar:

```
cd ..
```

Si queremos volver al home:

```
cd
```

Para ver que elementos existen en el directorio actual podemos usar el comando **ls**

```
ls
```

Para borrar una carpeta o elemento podemos usar el comando **rm -r**

```
rm -r nombre_elemento_a_borrar
```

Lo anterior creara la carpeta **ros2_ws** (nuestro espacio de trabajo), y dentro de ella la carpeta **src**, luego desde la carpeta **ros2_ws** lanzamos **colcon build**. Este comando creara las subcarpetas **build**, **install**, **log** en el directorio principal. En la carpeta **install** se encontrarán los archivos de configuración a los cuales realizaremos un **source** para poder utilizar este espacio de trabajo.



```
root@f5f86464c75a: ~/ros2_ws 80x22
root@f5f86464c75a:~# mkdir -p ~/ros2_ws/src
root@f5f86464c75a:~# cd ~/ros2_ws
root@f5f86464c75a:~/ros2_ws# colcon build

Summary: 0 packages finished [0.36s]
root@f5f86464c75a:~/ros2_ws# ls
build  install  log  src
root@f5f86464c75a:~/ros2_ws#
```

Ahora desde dentro de la carpeta src utilizamos el comando:

```
ros2 pkg create --build-type ament_python --license Apache-2.0 --node-name my_node my_package
```

ros2 pkg create package-name --build-type ament_python	Crea un paquete, debe ser llamado desde el directorio src de nuestro espacio de trabajo. (package-name es el nombre de nuestro paquete)
---	---

A partir de este comando se crea un paquete con el nombre elegido (en este caso es my_package) y un nodo de ejemplo llamado my_node.

Para este ejemplo, se utilizan argumentos opcionales --node-name y --license. La opción --node-name crea un ejecutable simple tipo Hello World en el paquete y --license declara la información de licencia para el paquete.

Lo siguiente que debemos realizar es construir (**build**) nuestros paquetes para esto desde la raíz del espacio de trabajo utilizamos el comando:

```
colcon build
```

```
root@f5f86464c75a: ~/ros2_ws 80x22
root@f5f86464c75a:~/ros2_ws# colcon build
Starting >>> my_package
Finished <<< my_package [2.63s]

Summary: 1 package finished [2.87s]
root@f5f86464c75a:~/ros2_ws#
```



Lo anterior construye todo nuestro espacio de trabajo, lo que puede llevar tiempo, podemos seleccionar que paquetes construir con la opción `--packages-select`, reduciendo el tiempo de trabajo:

```
colcon build --packages-select my_package
```

Luego de construir nuestro paquete podemos probar la ejecución del nodo de ejemplo para esto en una nueva consola, primero realizaremos el **source** a la instalación de ROS (underlay) y luego a nuestro espacio de trabajo (overlay):

```
root@f5f86464c75a: ~/ros2_ws 80x22
root@f5f86464c75a:~# source /opt/ros/jazzy/setup.bash
root@f5f86464c75a:~# cd ros2_ws/
root@f5f86464c75a:~/ros2_ws# source install/local_setup.bash
root@f5f86464c75a:~/ros2_ws#
```

Nota: Aquí el concepto de **underlay**, hace referencia a una capa base de elementos de ROS2, en nuestro caso la instalación base de ROS, pero podría haber sido otro espacio de trabajo. El **overlay**, aquí el espacio de trabajo que venimos de crear, este contiene paquetes propios, si el underlay tuviese los mismos paquetes (por ejemplo, en una versión de código distinta) los mimos serán ignorados y se utilizarán los del overlay en su lugar.

Con todo lo anterior lo único que nos falta es ejecutar el nodo de ejemplo para ver su funcionamiento, utilizando `ros2 run`:

```
root@f5f86464c75a: ~/ros2_ws 80x22
root@f5f86464c75a:~/ros2_ws# ros2 run my_package my_node
Hi from my_package.
root@f5f86464c75a:~/ros2_ws#
```

Observamos, como es de esperar en un programa tipo Hello world un mensaje por consola.

Examinando un poco el ejemplo podemos observar la estructura del paquete creada:



```
root@f5f86464c75a: ~/ros2_ws/src/my_package/my_package 80x14
root@f5f86464c75a:~/ros2_ws/src/my_package# ls
LICENSE  my_package  package.xml  resource  setup.cfg  setup.py  test
root@f5f86464c75a:~/ros2_ws/src/my_package# cd my_package/
root@f5f86464c75a:~/ros2_ws/src/my_package/my_package# ls
__init__.py  my_node.py
root@f5f86464c75a:~/ros2_ws/src/my_package/my_package#
```

Dentro de la carpeta del paquete podemos ver:

- **package.xml** conocido como manifiesto tendrá información útil de nuestro paquete y que podremos modificar a nuestra necesidad (autor, dependencias, utilidades de nuestro paquete, etc.). Abriéndolo con cualquier editor de texto podremos modificarlo:

```
package.xml X
ros2_ws > src > my_package > package.xml
1  <?xml version="1.0"?>
2  <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
3  <package format="3">
4    <name>my_package</name>
5    <version>0.0.0</version>
6    <description>TODO: Package description</description>
7    <maintainer email="root@todo.todo">root</maintainer>
8    <license>Apache-2.0</license>
9
10   <test_depend>ament_copyright</test_depend>
11   <test_depend>ament_flake8</test_depend>
12   <test_depend>ament_pep257</test_depend>
13   <test_depend>python3-pytest</test_depend>
14
15   <export>
16     <build_type>ament_python</build_type>
17   </export>
18 </package>
19
```

Se puede observar que los campos descripción y licencia contienen notas TODO. Esto se debe a que la descripción del paquete y la declaración de licencia no se configuran automáticamente, pero son necesarias si alguna vez desea compartir su paquete con la comunidad ROS. También deberíamos completar el campo maintainer.

Debajo de la etiqueta de licencia, verá algunos nombres de etiquetas que terminan en **_depend**. Aquí es donde su paquete.xml enumeraría sus dependencias en otros paquetes, para que colcon lo busque.

- **setup.py** el archivo setup.py contiene los mismos campos de descripción, mantenedor y licencia que package.xml, por lo que también debe configurarlos. Deben coincidir exactamente en ambos archivos. La versión y el nombre (nombre_paquete) también deben coincidir exactamente. Este archivo tendrá los puntos de entrada para nuestros nodos

(donde se buscaran los ejecutables). También podremos editarlo con cualquier editor de textos:

```
setup.py  X
ros2_ws > src > my_package > setup.py > ...
1  from setuptools import find_packages, setup
2
3  package_name = 'my_package'
4
5  setup(
6      name=package_name,
7      version='0.0.0',
8      packages=find_packages(exclude=['test']),
9      data_files=[
10         ('share/ament_index/resource_index/packages',
11          ['resource/' + package_name]),
12         ('share/' + package_name, ['package.xml']),
13     ],
14     install_requires=['setuptools'],
15     zip_safe=True,
16     maintainer='root',
17     maintainer_email='root@todo.todo',
18     description='TODO: Package description',
19     license='Apache-2.0',
20     tests_require=['pytest'],
21     entry_points={
22         'console_scripts': [
23             'my_node = my_package.my_node:main'
24         ],
25     },
26 )
27
```

- **carpeta my_package** dentro de esta carpeta encontraremos los archivos Python que conformaran el código de nuestro paquete, para el ejemplo **__init__.py** el constructor (para nuestro caso un archivo vacío) y **node.py** el código del nodo, que al igual que los anteriores podemos modificar a conveniencia con un editor de textos:

```
my_node.py  X
ros2_ws > src > my_package > my_package > my_node.py > main
1  def main():
2      print('Hi from my_package.')
3
4
5  if __name__ == '__main__':
6      main()
7
```



Actividad

- 1) Puede modificar el código del nodo Python recién visto para sacar un mensaje personalizado por pantalla y luego reconstruir el espacio de trabajo.



- **Programas de base**

En lo siguiente describiremos mediante ejemplos los códigos mínimos para los dos tipos de programas básicos de ROS utilizando tópicos: publicador y suscriptor.

Programa publicador:

```
import rclpy                                     #ROS Client Library for the Python Language.
from rclpy.node import Node                     #Importo clase Nodo

from std_msgs.msg import String #Importo tipo de mensajes que deseo utilizar en
mi nodo

#Creo clase de mi nodo
class MinimalPublisher(Node):

    #constructor de clase
    def __init__(self):
        super().__init__('minimal_publisher')
        self.publisher_ = self.create_publisher(String, 'topic123',
10)    #creo un publicador
        timer_period = 0.5 # seconds
        self.timer = self.create_timer(timer_period,
self.timer_callback)    #creo un timer
        self.i = 0

    #funcion del timer
    def timer_callback(self):
        msg = String()                                #creo mensaje de tipo cadena
        msg.data = 'Hola mundo: %d' % self.i          #cargo datos en mensaje
        self.publisher_.publish(msg)                  #utilizo publicador para enviar
el mensaje
        self.get_logger().info('Publicando: "%s"' % msg.data) #publico log en
consola de ejecucion
        self.i += 1

#funcion principal
def main(args=None):
    rclpy.init(args=args)

    minimal_publisher = MinimalPublisher()

    rclpy.spin(minimal_publisher)
```



```
# Destruimos el nodo en forma explicita
minimal_publisher.destroy_node()
rclpy.shutdown()

#Llamado a principal
if __name__ == '__main__':
    main()
```

Este código de ejemplo realizará publicaciones de una cadena de caracteres con la frase “Hola mundo” acompañada de un numero de publicación, en el tópico “topico123”.

Analizaremos paso a paso las líneas de código. En el encabezado vemos que ahora aparecen nuevas librerías:

```
import rclpy
```

Esta cargara la librería de Python de ROS, de la cual además incorporamos la clase nodo con la siguiente línea:

```
from rclpy.node import Node    #Importo clase Nodo
```

La siguiente línea:

```
from std_msgs.msg import String
```

Carga al tipo de mensaje que usaremos, en particular de la librería **std_msgs** donde se encuentran definidos los mensajes estándar de ROS (ver: https://docs.ros2.org/galactic/api/std_msgs/index-msg.html). Por cada nuevo tipo de mensaje que utilicemos deberemos cargar en el encabezado su librería/definición.

Continuando en el código encontramos la creación de una clase propia llamada MinimalPublisher, la cual hereda de la clase Nodo antes dicha. En su constructor encontramos:

```
super().__init__('minimal_publisher')
```

Esta línea, llama al constructor de la clase padre y lo que pasa como cadena es el nombre del nodo por defecto en nuestro caso minimal_publisher.

Además de lo anterior, en el constructor se crea un objeto publicador, quien será el responsable de realizar las publicaciones en el tópico indicado. La forma genérica para un publicador es la siguiente:



```
publisher_ = self.create_publisher(message_type, topic_name, queue_size)
```

- **message_type** es el tipo de mensaje (estructura de mensaje de ros) que deseamos publicar.
- **topic_name** es un string que contendrá el nombre del tópico en el que deseamos publicar
- **queue_size** es un entero que representa el tamaño de la cola de mensajes del publicador. En el caso en que nuestro programa genere mayor número de mensajes que lo que el publicador puede enviarlos, los mensajes más viejos en la cola son descartados.

Si nos interesara publicar en otro tópico con el mismo nodo solo debemos agregar otro objeto publicador.

Continuando en el código encontramos las líneas:

```
timer_period = 0.5 # seconds  
self.timer = self.create_timer(timer_period, self.timer_callback)
```

Estas líneas crean un objeto **timer** que gobernará la frecuencia con que se ejecutará el bucle principal (expresado en segundos a partir de la variable `timer_period` en este caso). Este objeto tendrá una función de callback definida más abajo en el código, donde encontraremos el código que se ejecutará en forma periódica.

```
#funcion del timer  
def timer_callback(self):  
    msg = String() #creo mensaje de tipo cadena  
    msg.data = 'Hola mundo: %d' % self.i #cargo datos en mensaje  
    self.publisher_.publish(msg) #utilizo publicador para  
    enviar el mensaje  
    self.get_logger().info('Publicando: "%s"' % msg.data) #publico log  
    en consola de ejecucion  
    self.i += 1
```

Dentro de la función de callback del timer, es interesante notar la construcción del mensaje, primero creamos el tipo de mensaje de ROS a utilizar (esto en realidad es una estructura, la cual contiene los tipos básicos de datos que enviaremos). Luego cargamos los datos de interés en la estructura y finalmente realizamos la publicación del mensaje con el método **publish** del objeto publicador.



Finalmente podemos analizar la función main del programa:

```
#funcion principal
def main(args=None):
    rclpy.init(args=args)

    minimal_publisher = MinimalPublisher()

    rclpy.spin(minimal_publisher)

# Destruimos el nodo en forma explicita
    minimal_publisher.destroy_node()
    rclpy.shutdown()
```

Aquí es donde sucede la “magia de ROS”, la función `rclpy.init(args=args)` inicia el nodo, luego instanciamos la clase antes explicada con la línea `minimal_publisher = MinimalPublisher()` y luego utilizamos el método **spin** con esta instancia.

Este último método realizará, utilizando la estructura de ros, que cada vez que la clase tenga algo para hacer (en nuestro caso cada vez que se tenga que ejecutar la función de callback del timer) se ejecute el código correspondiente, concluido esto “dormirá el nodo” (evitando ocupar un hilo de ejecución) hasta la próxima ejecución.

Finalmente, las líneas:

```
minimal_publisher.destroy_node()
rclpy.shutdown()
```

Destruyen el nodo y lo apagan.

Podemos copiar el código anterior en la carpeta `my_package` del ejemplo que venimos trabajando, bajo el nombre `publicador.py`.

Para correr este programa debemos antes realizar 3 cosas:

- 1) Agregar las dependencias en el archivo **package.xml** de nuestro paquete, para este caso serian agregar las líneas:

```
<exec_depend>rclpy</exec_depend>
<exec_depend>std_msgs</exec_depend>
```

Correspondientes a la librería de Python de ROS y a la de mensajes utilizados. En la siguiente imagen puede ver como quedaría este archivo, habiendo realizado las modificaciones en los metadatos antes nombrados:



```
publicador.py 3  package.xml  setup.py
ros2_ws > src > my_package > package.xml
1  <?xml version="1.0"?>
2  <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
3  <package format="3">
4      <name>my_package</name>
5      <version>0.0.0</version>
6      <description>Primeros pasos en ROS2</description>
7      <maintainer email="pepe@unlp.edu.ar">Pepe</maintainer>
8      <license>Apache-2.0</license>
9
10     <exec_depend>roscpp</exec_depend>
11     <exec_depend>std_msgs</exec_depend>
12
13     <test_depend>ament_copyright</test_depend>
14     <test_depend>ament_flake8</test_depend>
15     <test_depend>ament_pep257</test_depend>
16     <test_depend>python3-pytest</test_depend>
17
18     <export>
19         <build_type>ament_python</build_type>
20     </export>
21 </package>
```

- 2) Agregar un punto de entrada en el archivo **setup.py** (para que pueda encontrar nuestro nodo al intentar ejecutarlo) y los metadatos antes mencionados. En la siguiente imagen puede ver como quedaría el archivo, el punto de entrada se agregó en la línea 24, donde **talker** será el nombre ejecutable por el cual podremos llamar a nuestro nodo al utilizar **ros2 run**

```
publicador.py 3  package.xml  setup.py  X
ros2_ws > src > my_package > setup.py > ...
1  from setuptools import find_packages, setup
2
3  package_name = 'my_package'
4
5  setup(
6      name=package_name,
7      version='0.0.0',
8      packages=find_packages(exclude=['test']),
9      data_files=[
10          ('share/ament_index/resource_index/packages',
11           ['resource/' + package_name]),
12          ('share/' + package_name, ['package.xml']),
13      ],
14      install_requires=['setuptools'],
15      zip_safe=True,
16      maintainer='Pepe',
17      maintainer_email='pepe@unlp.edu.ar',
18      description='Primeros pasos en ROS2',
19      license='Apache-2.0',
20      tests_require=['pytest'],
21      entry_points={
22          'console_scripts': [
23              'my_node = my_package.my_node:main',
24              'talker = my_package.publicador:main',
25          ],
26      },
27  )
```

- 3) Construir nuestro espacio de trabajo mediante **colcon build**, recordar que este comando se debe lanzar desde la base de nuestro espacio de trabajo.



Para ejecutar nuestro nodo podemos abrir una nueva consola, realizar el **source** a la instalación de ROS y luego a nuestro espacio de trabajo y con el comando **ros2 run** llamar a nuestro código:

```
root@f5f86464c75a:~/ros2_ws# ros2 run my_package talker
[INFO] [1727029882.468396133] [pepito]: Publicando: "Hola mundo: 0"
[INFO] [1727029882.933936823] [pepito]: Publicando: "Hola mundo: 1"
[INFO] [1727029883.433835535] [pepito]: Publicando: "Hola mundo: 2"
[INFO] [1727029883.933754257] [pepito]: Publicando: "Hola mundo: 3"
[INFO] [1727029884.434591620] [pepito]: Publicando: "Hola mundo: 4"
[INFO] [1727029884.933881747] [pepito]: Publicando: "Hola mundo: 5"
[INFO] [1727029885.433869824] [pepito]: Publicando: "Hola mundo: 6"
[INFO] [1727029885.933895331] [pepito]: Publicando: "Hola mundo: 7"
[INFO] [1727029886.433867679] [pepito]: Publicando: "Hola mundo: 8"
[INFO] [1727029886.933981026] [pepito]: Publicando: "Hola mundo: 9"
[INFO] [1727029887.434313883] [pepito]: Publicando: "Hola mundo: 10"
[INFO] [1727029887.933851024] [pepito]: Publicando: "Hola mundo: 11"
[INFO] [1727029888.434056457] [pepito]: Publicando: "Hola mundo: 12"
```

Obtendremos que nuestro nodo realiza la publicación en el tópico deseado e imprime en consola el log indicado, podremos utilizar el comando **ros2 topic echo topico123** en otra consola para suscribirnos al tópico y escuchar sus publicaciones.

Programa suscriptor

A continuación, siguiendo el ejemplo del programa anterior, realizaremos un programa que se suscriba al tópico 'topico123' creado por el programa publicador y su vez muestre en su consola los datos recibidos. El código para este lo encontramos a continuación:

```
import rclpy                                     #ROS Client Library for the Python language.
from rclpy.node import Node                     #Importo clase Nodo

from std_msgs.msg import String                 #Importo tipo de mensajes que deseo utilizar
en mi nodo

#Creo clase de mi nodo
class MinimalSubscriber(Node):

    def __init__(self):
        super().__init__('minimal_subscriber')
        self.subscription = self.create_subscription(
            String,
            'topic123',
            self.listener_callback,
            10) #creo un suscriptor
        self.subscription # prevent unused variable warning

    #funcion del callback del suscriptor
    def listener_callback(self, msg):
        self.get_logger().info('Escuchando: "%s"' % msg.data)
```



```
#funcion principal
def main(args=None):
    rclpy.init(args=args)

    minimal_subscriber = MinimalSubscriber()

    rclpy.spin(minimal_subscriber)

    # Destruimos el nodo en forma explicita
    minimal_subscriber.destroy_node()
    rclpy.shutdown()

#llamado a principal
if __name__ == '__main__':
    main()
```

Aquí vemos una estructura similar al caso anterior solo algunas cosas a remarcar:

Vemos que en el constructor de la clase aparece un suscriptor, cuya forma genérica es:

```
sub= self.create_subscription(topic_type, topic_name, callback_function, queue_size);
```

Aquí los argumentos son:

- **topic_type** tipo de mensaje utilizado en el topico
- **topic_name** es el nombre del tópico al cual nos suscribiremos.
- **callback_function** función que ha de ejecutarse cuando un mensaje es recibido.
- **queue_size** largo de la cola de recepción de mensajes.

Respecto de la “**callback function**”, esta será una función genérica que tendrá la siguiente forma:

```
def callback_function_name(self, msg):
    #tu codigo
    #los datos recuperados del topico estan en la estructura msg del tipo de dato
    mensaje trabajado
```

El cuerpo de esta función tiene acceso a todos los campos del mensaje recibido y puede almacenar, usar o descartar la información contenida en él.

Luego el resto del código es similar al del publicador.

Para poder ejecutar este nodo debemos hacer las modificaciones correspondientes a los archivos package.xml y setup.py, para el primero como no agregamos ninguna nueva



dependencia no tendremos modificaciones, mientras que al segundo deberemos agregar el punto de acceso al nuevo nodo, el archivo quedaría como sigue:

```
publicador.py 3 package.xml setup.py x suscriptor.py 3
ros2_ws > src > my_package > setup.py > ...
1 from setuptools import find_packages, setup
2
3 package_name = 'my_package'
4
5 setup(
6     name=package_name,
7     version='0.0.0',
8     packages=find_packages(exclude=['test']),
9     data_files=[
10         ('share/ament_index/resource_index/packages',
11          ['resource/' + package_name]),
12         ('share/' + package_name, ['package.xml']),
13     ],
14     install_requires=['setuptools'],
15     zip_safe=True,
16     maintainer='Pepe',
17     maintainer_email='pepe@unlp.edu.ar',
18     description='Primeros pasos en ROS2',
19     license='Apache-2.0',
20     tests_require=['pytest'],
21     entry_points={
22         'console_scripts': [
23             'my_node = my_package.my_node:main',
24             'talker = my_package.publicador:main',
25             'listener = my_package.suscriptor:main',
26         ],
27     },
28 )
29
```

Para probar este código debemos construir nuestro espacio de trabajo y posteriormente correr ambos nodos, cada uno en una consola independiente, con los comandos:

```
ros2 run my_package talker
```

```
ros2 run my_package listener
```



Actividad :

A partir de los códigos de publicador y suscriptor antes mostrados:

- 1) Cree un nuevo paquete llamado `intro_unlp_robot`
- 2) Cree un nodo publicador que haga mover en círculos la tortuga del simulador `turtlesim`
- 3) Cree un nodo suscriptor que se suscriba el mensaje de posición del simulador y muestre en consola la componente lineal `x` e `y` de la velocidad.



Nota: Hasta aquí se realizó una muy pequeña introducción al mundo ROS, otros temas interesantes a explorar serían:

- Utilización de simuladores: Gazebo o Webots por ejemplo.
- Launchfiles: archivos que permiten lanzar múltiples nodos desde un único comando
- Bagfiles: archivos que permiten almacenar la información generada por el sistema (tópicos y servicios), y que posteriormente puede ser reproducida – ideal para el testeado de códigos.